

Cloud Agents

Best Practices

Use these recommendations to get more reliable Cloud Agent runs.

Set up the environment first

Use [Cloud agent setup](#) so that Cursor has its environment configured. Like a human developer, Cursor does better work if its environment is set up correctly.

Ensure the agent can access what it needs

Before running a Cloud Agent, verify these prerequisites:

- **Secrets:** Make sure the agent has access to required secrets (API keys, database credentials, etc.) through the [Secrets tab](#) in your dashboard.
- **Egress controls:** If you have [network access](#) restrictions enabled, ensure all URLs your local development requires are whitelisted.
- **Local testability:** Your repo should be set up to run well locally without requiring external services that cannot be reached from a VM. If it is hard for a human developer to test locally, it will also be hard for an agent.

Use skills and agents.md to configure your agent

If the cloud agent is having difficulty testing its changes, we recommend using [skills](#) and [agents.md](#) to configure your agent.

Think of the agent as a smart, but low-context human developer. The best way to make sure it does the right thing is to give it the context it needs to understand what to do.

For example, at Cursor our [agents.md](#) lists tips for running and debugging the most commonly used microservices in our mono-repo. We also have lots of skills about how to test and debug key services, each with clear instructions on when to use the skill.

The skills contain in-depth details, such as how to debug a specific microservice or how to set up a third-party dependency when needed for testing.

Give the agent the tools it needs

We have often found that agents are limited by the tools they have access to. We recommend using [MCP](#) and creating custom tools so that the agent has access to the same systems a human developer would.

Mold the tools to the agent

It is important to create tools that the agent is good at using. We recommend creating tools, and iterating based on observations of how the agent uses them.

For example, at Cursor we have created a custom CLI for the model to run micro-services in our codebase. We found that when running custom dev commands, e.g. from a `package.json` file, some models would forget arguments, or agents would get distracted by noisy build logs which human developers knew to ignore.

